

The Complete Backend & Model Walkthrough

Part 1: What Problem Are We Actually Solving?

A pathologist looks at biopsy slides under a microscope (or, in modern hospitals, as a scanned digital image) to decide: is this tissue cancerous or not? In India, there aren't enough pathologists for how many slides need reviewing. So slides pile up in a queue and get reviewed in whatever order they arrived — not in order of how urgent they are.

Our idea: build an AI system that looks at a slide first, and tells the hospital "review this one immediately, it looks serious" or "this one can wait, it looks fine" — and, critically, "we're not sure about this one, a senior doctor needs to double check it personally."

That's it. That's the whole point. Everything below is just *how* we built a computer program that can look at a slide image and make that call.

Part 2: What Is a "Whole Slide Image" (WSI)?

When a lab scans a biopsy sample, they don't get a normal photo. They get a **gigantic image** — imagine a photo so detailed that if you zoomed in, you could see individual cells. Our slides were roughly **97,792 × 221,184 pixels**. For comparison, a normal phone photo is maybe 4000×3000 pixels. This WSI is thousands of times bigger.

Why does size matter? You cannot feed an image that huge into any AI model — it wouldn't fit in memory, and there's no AI architecture designed to look at something that big directly. So the first problem we had to solve was: how do you get a computer to "look at" an image that's too big to look at?

Part 3: Step One — Cutting the Slide Into Small Pieces ("Tiling")

We chop the giant slide into a grid of small square images, each **256×256 pixels**. Think of it like cutting a huge poster into hundreds of small index cards, each showing a tiny patch of the original picture.

Each one of these small squares is called a **"patch."**

Problem: a slide this big could be cut into tens of thousands of patches. Most of that space is just empty glass — there's no tissue there at all, it's blank. We don't want to waste time processing thousands of blank white squares.

Part 4: Step Two — Finding Where the Actual Tissue Is ("Tissue Masking")

We use a technique called **Otsu thresholding**. Here's the plain-English version:

- We take a small, low-resolution "thumbnail" version of the whole slide (like a quick preview).
- Tissue (the biological material) shows up darker/more colored in this thumbnail. Empty glass shows up bright white.
- Otsu's method is a simple math trick that automatically finds the best brightness cutoff point to separate "this looks like tissue" from "this looks like empty background" — without a human manually deciding the cutoff.

- We then only keep patches that fall in the "this is tissue" zone.

On our slides, this cut down anywhere from ~82,000 possible grid positions to somewhere between 1,900 and 6,000 tissue-containing positions — a big reduction, saving huge amounts of processing time.

We then took a sample of up to **400 patches** per slide from that tissue-positive area, spread evenly across it, to keep things a consistent, manageable size across all 20 slides.

Part 5: Step Three — Fixing Color Differences ("Stain Normalization")

Biopsy slides are stained with chemical dyes (called **H&E staining** — Hematoxylin and Eosin) so cell structures become visible under a microscope. Hematoxylin stains cell nuclei purple/blue; Eosin stains everything else pink.

Problem: different hospitals, different scanning machines, and different staining batches produce slightly different colors for the *same* biological tissue. If we don't fix this, our AI might get confused and think color differences mean something biologically important, when really it's just "this hospital's scanner has a yellowish tint."

Solution: Macenko normalization. This is a mathematical technique that:

1. Looks at a patch's colors and mathematically "un-mixes" them into their two underlying dye components (how much Hematoxylin, how much Eosin).
2. Remaps those components to match a standard reference color profile.
3. Reconstructs the image with standardized colors.

We used a library called torchstain to do this math for us. We picked one "reference" patch, calibrated the normalizer to it once, and then applied that same calibration consistently to every other patch across all 20 slides — so everything is color-standardized the same way.

Part 6: Step Four — Turning Images Into Numbers ("Feature Extraction")

Here's a core AI concept: neural networks don't work directly with pictures the way our eyes do. They work with **numbers**. So we need to convert each 256×256 pixel patch into a list of numbers that captures "what's important about this image."

How: we use a pre-existing, already-trained neural network called **ResNet-50**. This network was originally trained by researchers on 1.2 million random photos (dogs, cars, buildings, everyday objects — nothing medical) to learn to recognize general visual patterns: edges, textures, shapes, colors.

We're not asking ResNet-50 to tell us "is this cancer." We're using it as a **feature extractor** — we run our patch through it, and instead of taking its final "this is a dog/cat/car" answer, we grab the numbers from a layer *just before* that final decision. That layer outputs **2048 numbers** per patch. Those 2048 numbers are a compressed numerical summary of everything visually distinctive about that patch — shapes, textures, colors, patterns — even though the network was never trained on medical images specifically.

Why not train our own network from scratch? We only have 8,000 patches total (from 20 slides). That's nowhere near enough data to train a good image-recognition network from zero — it would just memorize our tiny dataset instead of learning general patterns. Reusing a network already trained on 1.2 million images (called "**transfer learning**") is much more data-efficient. This is standard practice in this exact research field.

Result: every slide now becomes not one giant image, but a collection of up to 400 patches, each turned into a list of 2048 numbers. We save this to a small file per slide (a .npz file, roughly 1-1.5 MB) and can now delete the giant original image (which was 1-3 GB) — we don't need the pixels anymore, just the extracted numbers.

Part 7: Step Five — The Actual AI Model ("Multiple Instance Learning")

Now here's where it gets interesting. We have a slide represented as **up to 400 separate patches**, each with its own 2048 numbers. But we only have **one label** per slide: "malignant" or "benign." We don't know *which specific patches* contain the cancer — a pathologist would have to manually mark that, and we don't have that level of detailed labeling.

This is called a "**weakly-supervised**" problem, and the technique we use to solve it is called **Multiple Instance Learning (MIL)**.

The core idea, explained with an analogy: imagine a bag containing 400 marbles. You're told "this bag is either 'special' or 'normal' overall" — but you don't know which individual marbles (if any) are special. MIL is a method for training a model that looks at all 400 marbles together, decides how much attention to pay to each one, and produces one overall "special or not" answer for the whole bag — while, as a side effect, also telling you which marbles it thought were most important.

In our case: the "bag" is the slide, the "marbles" are the 400 patches (each represented by its 2048 numbers).

How the model does this — the "Attention" mechanism:

1. Every patch's 2048 numbers get passed through a small neural network layer that shrinks them down to a smaller internal representation (256 numbers).
2. A separate small "attention" network looks at each patch's representation and outputs **one single number per patch** — essentially, "how important is this specific patch for making the final decision?"
3. These importance scores are normalized (using a math function called **softmax**) so they all add up to 1 — like turning raw scores into percentages of total attention.
4. The model then computes a **weighted average** of all 400 patches' representations, using those attention scores as the weights. Patches the model considers unimportant contribute almost nothing to the final average; patches it considers important contribute a lot.
5. This weighted average — one single representation for the whole slide — gets passed through a final small classifier layer that outputs one number: the probability the slide is malignant.

This is genuinely elegant because: the model automatically learns which patches matter *without ever being told*. It only ever sees "this whole slide is malignant" or "this whole slide is benign" during

training, and through that weighted-attention mechanism, it figures out on its own which patches were probably responsible for that label. Those attention scores are also exactly what we display as the colorful heatmap in the dashboard — bright/hot colors = "the model paid a lot of attention to this patch."

This specific architecture (patch projection → gated attention → weighted average → classify) is called **ABMIL** (Attention-Based Multiple Instance Learning), based on a well-known 2018 research paper, with some refinements from a follow-up paper called **CLAM**.

Part 8: How the Model "Learns" (Training)

The model starts with random, meaningless internal numbers (called **parameters** — our model has about 623,000 of them). Training is the process of gradually adjusting those numbers so the model's predictions get closer to being correct.

The training loop, step by step:

1. Show the model one training slide (its 400 patches).
2. The model produces a probability (e.g., "73% malignant").
3. Compare that to the *actual* correct answer (0 = benign, 1 = malignant) using a mathematical formula called **loss** (specifically, "binary cross-entropy") — this loss number is small if the prediction was close to correct, and large if it was very wrong.
4. Using calculus (**backpropagation** — the standard method all neural networks use), the computer figures out exactly how to nudge each of the 623,000 internal numbers to slightly reduce that loss.
5. Repeat this for every slide in the training set.
6. Do the *entire* process again, multiple times over — each full pass through all training slides is called an **epoch**. We did this **25 times** (25 epochs).

Important clarification (this answers your teammate's earlier question): the model doesn't learn "slide by slide, permanently getting better as it goes and never returning to earlier slides." It sees ALL training slides repeatedly, in a different random order each epoch, over and over across those 25 rounds. By the end, its internal numbers reflect having learned from every training slide roughly equally — not "the last slide it saw" or "worse understanding of the first slide."

Part 9: Why We Split Data Into Train / Validation / Test

We had 20 real slides total. We didn't use all 20 for training. We split them:

- **12 slides → Training set:** the model actually learns from these, adjusting its internal numbers based on them.
- **4 slides → Validation set:** used during development to check "is the model actually getting better, or is something wrong," without ever updating the model's numbers based on these.

- **4 slides → Test set:** these are set aside and *never shown to the model at all* during training. This is our one true, honest measure of "does this actually work on slides it's never seen before."

Why this split matters so much: if we only reported how well the model does on the *training* slides, that number would be misleading — much like a student who's seen the exact exam questions beforehand. Our model actually gets a *perfect* score (100%) on training slides, but that's expected and not impressive — it's had 25 rounds of exposure to memorize patterns specific to those exact 12 slides. The test set score (which came out to **75%**, explained more below) is the real, honest answer to "would this work on a brand new slide."

One more important detail — the split happens at the "slide" level, not the "patch" level. All 400 patches belonging to one slide always stay together in the same set (all in training, or all in test, never split across both). This matters because patches from the same slide are similar to each other (same patient, same staining batch) — if we let some patches from one slide leak into training while others from that same slide were in the test set, the model could "cheat" by recognizing that specific slide's characteristics rather than genuinely learning to detect cancer.

Part 10: How We Measure "Accuracy" (AUC, not raw accuracy)

We don't use simple "percent correct" as our main metric. We use something called **AUC (Area Under the ROC Curve)**.

Why not simple accuracy? Imagine 90% of our slides were benign. A lazy model could just always guess "benign" and be 90% "accurate" while being completely useless — it never actually detects cancer. AUC avoids this trap.

What AUC actually measures, in plain terms: if you randomly picked one malignant slide and one benign slide from our results, AUC is the probability that the model correctly ranked the malignant one as *more* likely to be cancerous than the benign one. A perfect model scores 1.0 (always ranks correctly). Random guessing scores 0.5 (a coin flip).

Our actual results:

- Training set AUC: **1.00** (perfect — but expected/not impressive, as explained above — this reflects memorization on data it directly learned from)
- Validation set AUC: **0.75**
- Test set AUC: **0.75** (this is our real, honest, headline number for the real-data pipeline)

An AUC of 0.75 with only 4 test slides means: of the 4 possible ways to pair up a malignant slide with a benign slide and check if the model ranked them correctly, it got 3 out of 4 right.

Part 11: The Uncertainty Layer — Our Key Differentiator ("MC-Dropout")

Every MIL model discussed above (plain ABMIL, CLAM, published research) only outputs one number: a probability. The problem: **a confident-sounding wrong answer is dangerous in medicine.** If the model says "12% malignant" and is actually wrong, a doctor might trust that number and deprioritize a slide that actually needed urgent attention.

Our addition: we want the model to also tell us *how confident it actually is* — not just spit out a probability.

How MC-Dropout works, explained simply:

- **Dropout** is a normal AI training trick where, during training, the model randomly "turns off" some of its internal connections each time, forcing it to not over-rely on any single pathway. Normally, dropout is switched *off* once training is done, and the model gives one clean, consistent answer at prediction time.
- **Our trick (Monte Carlo Dropout):** we deliberately leave dropout *on* even after training, and instead of asking the model once, we ask it the same question **20 separate times** in a row for the same slide. Because dropout randomly disables different connections each time, we get 20 *slightly different* predictions for the same slide.
- **If those 20 predictions are all close together** (e.g., 89%, 91%, 90%, 88%...) → the model is confident, regardless of whether the probability itself is high or low.
- **If those 20 predictions are scattered widely** (e.g., 30%, 75%, 45%, 60%...) → the model itself doesn't really know, even if the *average* of those numbers looks moderate.
- We take the **average** of the 20 runs as our final probability, and the **standard deviation** (a measure of how spread out the 20 numbers were) as our **uncertainty score**.

This is a real, established statistical technique for approximating a neural network's confidence, not something we invented from scratch — but applying it specifically to build a clinical triage/routing system is our own contribution on top of it.

Part 12: The Triage Router — Turning Numbers Into a Decision

This is the actual "product" — the part that makes this more than just a research classifier.

Every slide gets sorted into one of three tiers based on both its probability AND its uncertainty score:

Tier	Rule	Meaning
Tier 1 — Urgent	Probability > 70% AND uncertainty is low	Model is confident it's malignant — review immediately
Tier 2 — Routine	Probability ≤ 30% AND uncertainty is low	Model is confident it's benign — normal queue
Tier 3 — Uncertain/Flagged	Uncertainty is high, regardless of the probability number	Model isn't sure — a senior pathologist MUST personally review this, no shortcuts

Why this is the actual innovation: every other MIL system just gives a doctor a probability and lets them figure out how much to trust it. Ours explicitly tells the workflow system "this one needs a human expert, don't trust the automated number here" — which is a fundamentally safer way to deploy AI in a medical setting.

Part 13: Why Real-World Results Look Different From Our "Cleaner" Synthetic Test

We actually built and tested this system on **two different datasets**, and it's important your teammates understand why the numbers differ:

1. Synthetic (fake, computer-generated) data: we artificially generated fake patch numbers designed to mimic what real cancer/normal tissue differences might look like, with adjustable difficulty. We tuned this to be realistically difficult (not too easy, not impossibly hard), tested it on 550 fake "slides," and got a Test AUC of **0.86**, with **14% of slides correctly flagged as "uncertain."** This proves our architecture and uncertainty mechanism work correctly, at a larger scale, in a controlled setting we designed ourselves.

2. Real CAMELYON16 data: actual real breast cancer biopsy slides from an internationally recognized public research dataset, used by the original MIL research papers we're building on. Only 20 real slides (due to storage/time constraints — each slide is 1-3 GB and takes minutes to download and process). Test AUC came out to **0.75**.

Why real data performed worse, honestly explained: with only 12 training slides, our model (with 623,000 adjustable internal numbers) can very easily just memorize those 12 examples rather than learning patterns that generalize — this is called **overfitting**, and it's expected with such a small dataset, not a flaw unique to our approach. We also found a specific, real pattern in *why* it underperformed: two malignant test slides (tumor_007, tumor_010) contain what's called **micrometastasis** — a very tiny cluster of cancer cells (sometimes just 1-3 out of 400 sampled patches show any cancer at all). With so few training examples of malignant tissue, the attention mechanism hasn't reliably learned to notice such a small signal among hundreds of normal-looking patches, so it under-predicted those two cases. Meanwhile, slides with larger, obvious tumor regions (**macrometastases**) were correctly identified with high confidence (79-99% probability).

This is actually a good, honest story for the eval — it shows we tested on real data, found a real, well-documented limitation from the field, and understand exactly why it happened rather than pretending everything is perfect.

Part 14: The Final Output — How It All Connects to the Dashboard

1. pipeline.py runs the entire process above (train the model, run MC-Dropout inference, route into tiers) and exports one big **JSON file** — a structured text file listing, for every slide: its probability, uncertainty score, assigned tier, how many patches it had, and the coordinates + attention weight of every single patch.
2. That JSON file gets copied into our **React** (JavaScript) frontend dashboard's folder.
3. The dashboard reads that file and displays: a sortable queue of all slides sorted by tier, and a detail page per slide showing the probability, uncertainty, and a **visual heatmap** — literally drawing each patch as a small colored square, colored according to its attention weight (dark = low attention, bright yellow/orange = high attention) — so a pathologist can visually see exactly which regions of the tissue the AI focused on.
4. We built a toggle so the dashboard can show either our real CAMELYON16 results or our synthetic benchmark results, since both tell a different, complementary part of the story.

Quick Summary, If Someone Only Has 60 Seconds

We take a giant cancer biopsy image → cut it into hundreds of small tiles → fix color inconsistencies → convert each tile into numbers using a pretrained AI vision network → feed all those tiles into a custom attention-based model that learns which tiles matter and produces one malignancy score for the whole slide → run that same model 20 times with a randomness trick to measure how *confident* it actually is → use both the score AND the confidence to sort every slide into "urgent," "routine," or "needs a human expert right now" → display all of this in a dashboard a pathologist would actually use.